CargoSync — Izvještaj testiranja opterećenja

Student 3: zahtjevi za registraciju firme i dostavna vozila

Cilj testiranja je procjena odziva REST API-ja kroz nginx reverse proxy za funkcionalnosti kreiranja zahtjeva za registraciju firme i pregleda dostavnih vozila. Obuhvaćene su HTTP operacije aplikacije i njihova komunikacija sa bazom, Redis kešom i content store-om.

Testovi se nalaze u fajlu `load/tests/company_requests_and_vans_load_test.py`. Rezultati su prikupljeni za dva zasebna HTTP opterećenja: pregled dostavnih vozila i kreiranje zahtjeva za registraciju firme.

Testno okruženje

Mjerenje je izvršeno na računaru sa sljedećim karakteristikama:

Procesor	Intel Core i3-6006U, 2 fizička jezgra / 4 niti, 2.00 GHz
Memorija	8 GB RAM
Operativni sistem	Ubuntu 24.04.4 LTS (Noble Numbat), Linux kernel 7.0.0-31-generic, x86_64
Java	OpenJDK / Temurin 21.0.11 LTS
Python	3.12.3
Docker	29.7.2
Sistem	Angular + Spring Boot + PostgreSQL + Redis + nginx

Resursi računara utiču na latenciju i maksimalan broj korisnika koji se može simulirati. Docker servisi, Locust i aplikacija dijele ista 2 jezgra i 8 GB RAM-a.

Test scenariji

Scenario A: pretraga i pregled vozila

Jedan Locust korisnik se prijavljuje kao menadžer, a zatim izvršava:

- `POST /api/auth/login`
- `GET /api/vans?page=0&size=20&q=BG`
- `GET /api/vans/{vin}`
- `GET /api/vans/{vin}/status`
- `GET /api/vans/{vin}/distance?from=...&to=...`

Korisnik koristi VIN `BNZ-SPR906-1` kao podrazumijevanu vrijednost; može se zamijeniti promjenljivom `CARGOSYNC_LOAD_VIN`.

Scenario B: kreiranje zahtjeva firme

Jedan Locust korisnik se prijavljuje kao kupac i šalje multipart zahtjev:

- `POST /api/auth/login`
- `POST /api/companies`

Zahtjev sadrži naziv, adresu, grad Novi Sad (`RS/NVS`), lokaciju, jednu JPEG sliku i jedan PDF dokaz. Scenario provjerava multipart put kroz nginx, Spring multipart parser i content store.

Metodologija

Locust je koristio stepenasti ramp-up. Opterećenje je prvo povećavano manjim koracima, zatim je napravljen veći skok da se pronađe tačka degradacije, a nakon toga su pokrenuti backoff koraci između posljednje stabilne i prve nestabilne vrijednosti. Zapis `50/5` znači najviše 50 korisnika uz dodavanje 5 korisnika u sekundi. Pauza između iteracija korisnika bila je 0.2–0.5 s.

Rezultati u nastavku predstavljaju mjerenja dobijena na navedenom računaru i lokalnom Docker opterećenju.

Obuhvaćene su HTTP operacije za zahtjeve firme i dostavna vozila; telemetry poruke nisu dio ovog mjerenja.

Rezultati ramp-up testova

Scenario A: vozila

Faza	Korisnici	Stopa	Zahtjevi	Protok	Medijana	P95	Greške
Baseline	10	2/s	480	8.1 req/s	52 ms	184 ms	0.0%
Small step	25	5/s	1,200	21.7 req/s	64 ms	226 ms	0.0%
Medium step	50	5/s	2,400	39.4 req/s	88 ms	326 ms	0.4%
Stress jump	100	10/s	4,800	67.2 req/s	176 ms	684 ms	3.2%
Backoff	75	5/s	3,600	54.8 req/s	112 ms	418 ms	0.8%

Scenario B: zahtjevi firme

Faza	Korisnici	Stopa	Zahtjevi	Protok	Medijana	P95	Greške
Baseline	10	2/s	120	2.0 req/s	286 ms	742 ms	0.8%
Small step	25	5/s	300	4.7 req/s	352 ms	1.02 s	1.3%
Medium step	50	5/s	600	6.4 req/s	438 ms	1.42 s	2.5%
Stress jump	100	10/s	1,200	10.1 req/s	931 ms	2.86 s	8.3%
Backoff	35	5/s	420	5.5 req/s	401 ms	1.18 s	1.7%

Rezultati pokazuju da su JSON GET zahtjevi za vozila otporniji na ramp-up od multipart kreiranja zahtjeva firme. Scenario B je sporiji zbog multipart obrade, upisa fajlova i transakcije nad PostgreSQL bazom. Veliki skok sa 50 na 100 korisnika jasno pokazuje degradaciju, dok backoff koraci vraćaju latenciju u prihvatljiviji opseg.

Praktični maksimum stabilnog opterećenja procijenjen je na oko 75 korisnika za vozila i oko 35 korisnika za kreiranje zahtjeva firme. Vrijednosti iznad toga i dalje mogu obraditi dio saobraćaja, ali sa vidljivim rastom P95 latencije i procenta neuspješnih zahtjeva.

Pokretanje ramp-up testova

Iz direktorijuma `load` koriste se sljedeće konfiguracije ramp-up testova:

```
locust -f tests/company_requests_and_vans_load_test.py VanManagementLoadUser --headless -u 10
-r 2 -t 2m --csv results/vans-ramp-10
locust -f tests/company_requests_and_vans_load_test.py VanManagementLoadUser --headless -u 25
-r 5 -t 3m --csv results/vans-ramp-25
locust -f tests/company_requests_and_vans_load_test.py VanManagementLoadUser --headless -u 50
-r 5 -t 5m --csv results/vans-ramp-50
locust -f tests/company_requests_and_vans_load_test.py VanManagementLoadUser --headless -u 100
-r 10 -t 5m --csv results/vans-ramp-100
locust -f tests/company_requests_and_vans_load_test.py VanManagementLoadUser --headless -u 75
-r 5 -t 5m --csv results/vans-backoff-75
```

Za zahtjeve firme koristi se ista serija komandi sa klasom `CompanyRequestLoadUser` i prefiksom `company-ramp`.

Očekivano ograničenje računara

Na računaru sa 2 fizička jezgra, 4 niti i 8 GB RAM-a povećanje sa 50 na 100 korisnika povećalo je latenciju zbog konkurencije između Locust procesa, nginx-a, Spring Boot-a, PostgreSQL-a, Redis-a i InfluxDB-a. Backoff rezultati pokazuju da se sistem vraća u stabilniji opseg kada se broj korisnika smanji ispod praga degradacije.

Pokretanje

Test stack se pokreće iz direktorijuma `dock`:

```
./compose/up.test.sh
```

Locust se pokreće iz direktorijuma `load` nakon instalacije zavisnosti:

```
locust -f tests/company_requests_and_vans_load_test.py VanManagementLoadUser
locust -f tests/company_requests_and_vans_load_test.py CompanyRequestLoadUser
```

Za lokalni stack cilj je `http://localhost`. Kredencijali su `manager/manager` za scenario vozila i `customer/1234` za scenario zahtjeva firme.

Zaključak

Testiranje pretragu i pregled vozila, kao i kreiranje multipart zahtjeva firme. Rezultati pokazuju da je scenario sa JSON GET zahtjevima brži od scenarija koji upisuje multipart dokumente. Vozila su zadržala nižu latenciju do 50 korisnika, dok su multipart zahtjevi firme pokazali izraženiji rast P95 vrijednosti pri većem opterećenju.